

Ruby is:

1. **High-Level:** (reading and writing Ruby is really easy—it looks a lot like regular English)
2. **Interpreted:** You don't need a compiler to write and run Ruby.
3. **Object-Oriented:** It allows users to manipulate data structures called objects in order to build and execute programs. Everything in Ruby is an object.

Ruby was designed by **Yukihiro Matsumoto** (often just called "Matz") in 1995*.

Matz set out to design a language that emphasized human needs over those of the computer, which is why Ruby is so easy to pick up.

Data Types

Three Datatypes in Ruby

1. Numeric (any number)
2. Boolean (which can be true or false)
3. String (words or phrases like "I'm learning Ruby!")

Reminder: Never use quotation marks (' or ") with booleans, or Ruby will think you're talking about a string instead of a value that can be true or false.

```
my_num = 25    # Add your code here!

my_boolean = true    # And here!

my_string = "Ruby"    # Also here.

puts my_num
puts my_boolean
puts my_string
```

You are using `puts my_num` to print the value of the variable `my_num` to the screen.

Variables

Declaring variables in Ruby is easy: you just write out a name like `my_num`, use `=` to assign it a value, and you're done!

A variable is a label that Ruby assigns to a particular object. There are four different types of variables in Ruby. Each type starts with a special character to indicate the type of variable it is.

Local Variables

Local variables must begin with a lowercase letter or `_`. These variables are local to the code block of the method they are declared in.

```
color = "green"
_person = "Shannon"
```

Instance Variables

Instance variables begin with an `@` symbol. Instance variables are variables that belong to an object.

```
class Person
  def initialize(first_name, last_name, age, gender)
    @first_name = first_name
    @last_name = last_name
    @age = age
    @gender = gender
  end
  def person_info
    puts "Name: #{@first_name} #{@last_name}"
    puts "Age: #{@age}"
    puts "Gender: #{@gender}"
  end
end

person1 = Person.new("John", "Doe", 30, "Male")
person2 = Person.new("Jane", "Doe", 25, "Female")

person1.person_info()
person2.person_info()
```

Class Variables

Class variables begin with an `@@` sign. Class variables are available across different objects shared by all the descendants of the class. They must be initialized before use.

```
class Person2
  # Class variable to keep track of the number of instances
  @@number_of_persons = 0
  def initialize(first_name, last_name, age)
    @first_name = first_name
    @last_name = last_name
    @age = age
  end

  def display_full_name
    puts "Full Name: #{@first_name} #{@last_name} #{@age} years old"
  end
end
```

```
end

def total_number_of_persons
  @@number_of_persons += 1
  puts "Total Number of Persons: #{@number_of_persons}"
end
end

person1 = Person2.new("Alice", "Smith", 28)
person1.display_full_name()
person1.total_number_of_persons()
person2 = Person2.new("Bob", "Johnson", 35)
person2.display_full_name()
person2.total_number_of_persons()
```

Global Variables

Global variables begin with an \$ symbol. While Class variables are not available across different classes, global variables are. Its scope is global, meaning that it can be accessed from anywhere in the program.